

第7章 暴力求解法

学习目标

- ☑ 掌握整数、子串等简单对象的枚举方法
- ☑ 熟练掌握排列生成的递归方法
- ☑ 熟练掌握用“下一个排列”枚举全排列的方法
- ☑ 理解解答树，并能估算典型解答树的结点数
- ☑ 熟练掌握子集生成的增量法、位向量法和二进制法
- ☑ 熟练掌握回溯法框架，并能理解为什么它往往比生成-测试法高效
- ☑ 掌握回溯法的常见优化方法
- ☑ 熟练掌握八数码问题的 BFS 实现，包括结点查找表的哈希实现和 STL 集合实现
- ☑ 熟练掌握埃及分数问题的 IDA*实现

很多问题都可以“暴力解决”——不用动太多脑筋，把所有可能性都列举出来，然后一一试验。尽管这样的方法显得很“笨”，但却常常是行之有效的。

7.1 简单枚举

在枚举复杂对象之前，先尝试着枚举一些相对简单的内容，如整数、子串等。尽管暴力枚举不用太动脑筋，但对问题进行一定的分析往往会让算法更加简洁、高效。

提示 7-1：即使采用暴力法求解问题，对问题进行一定的分析往往会让算法更简洁、高效。

例题 7-1 除法 (Division, UVa 725)

输入正整数 n ，按从小到大的顺序输出所有形如 $abcde/fghij = n$ 的表达式，其中 $a \sim j$ 恰好为数字 0~9 的一个排列（可以有前导 0）， $2 \leq n \leq 79$ 。

样例输入：

62

样例输出：

79546 / 01283 = 62

94736 / 01528 = 62

【分析】

枚举 0~9 的所有排列？没这个必要。只需要枚举 $fghij$ 就可以算出 $abcde$ ，然后判断是否所有数字都不相同即可。不仅程序简单，而且枚举量也从 $10! = 3628800$ 降低至不到 1 万，

而且当 abcde 和 fghij 加起来超过 10 位时可以终止枚举。由此可见,即使采用暴力枚举,也是需要认真分析问题的。

例题 7-2 最大乘积 (Maximum Product, UVa 11059)

输入 n 个元素组成的序列 S , 你需要找出一个乘积最大的连续子序列。如果这个最大的乘积不是正数, 应输出 0 (表示无解)。 $1 \leq n \leq 18, -10 \leq S_i \leq 10$ 。

样例输入:

```
3
2 4-3
5
2 5 -1 2 -1
```

样例输出:

```
8
20
```

【分析】

连续子序列有两个要素: 起点和终点, 因此只需枚举起点和终点即可。由于每个元素的绝对值不超过 10 且不超过 18 个元素, 最大可能的乘积不会超过 10^{18} , 可以用 long long 存储。

例题 7-3 分数拆分 (Fractions Again?!, UVa 10976)

输入正整数 k , 找到所有的正整数 $x \geq y$, 使得 $\frac{1}{k} = \frac{1}{x} + \frac{1}{y}$ 。

样例输入:

```
2
12
```

样例输出:

```
2
1/2 = 1/6 + 1/3
1/2 = 1/4 + 1/4
8
1/12 = 1/156 + 1/13
1/12 = 1/84 + 1/14
1/12 = 1/60 + 1/15
1/12 = 1/48 + 1/16
1/12 = 1/36 + 1/18
1/12 = 1/30 + 1/20
1/12 = 1/28 + 1/21
1/12 = 1/24 + 1/24
```

【分析】

既然要求找出所有的 x, y ，枚举对象自然就是 x, y 了。可问题在于，枚举的范围如何？从 $1/12=1/156+1/13$ 可以看出， x 可以比 y 大很多。难道要无休止地枚举下去？当然不是。由于 $x \geq y$ ，有 $\frac{1}{x} \leq \frac{1}{y}$ ，因此 $\frac{1}{k} - \frac{1}{y} \leq \frac{1}{y}$ ，即 $y \leq 2k$ 。这样，只需要在 $2k$ 范围之内枚举 y ，然后根据 y 尝试计算出 x 即可。

7.2 枚举排列

有没有想过如何打印所有排列呢？输入整数 n ，按字典序从小到大的顺序输出前 n 个数的所有排列。前面讲过，两个序列的字典序大小关系等价于从头开始第一个不相同位置处的大小关系。例如， $(1, 3, 2) < (2, 1, 3)$ ，字典序最小的排列是 $(1, 2, 3, 4, \dots, n)$ ，最大的排列是 $(n, n-1, n-2, \dots, 1)$ 。 $n=3$ 时，所有排列的排序结果是 $(1, 2, 3)$ 、 $(1, 3, 2)$ 、 $(2, 1, 3)$ 、 $(2, 3, 1)$ 、 $(3, 1, 2)$ 、 $(3, 2, 1)$ 。

7.2.1 生成 $1 \sim n$ 的排列

我们尝试用递归的思想解决：先输出所有以 1 开头的排列（这一步是递归调用），然后输出以 2 开头的排列（又是递归调用），接着是以 3 开头的排列……最后才是以 n 开头的排列。

以 1 开头的排列的特点是：第一位是 1，后面是 $2 \sim 9$ 的排列。根据字典序的定义，这些 $2 \sim 9$ 的排列也必须按照字典序排列。换句话说，需要“按照字典序输出 $2 \sim 9$ 的排列”，不过需注意的是，在输出时，每个排列的最前面要加上“1”。这样一来，所设计的递归函数需要以下参数：

- 已经确定的“前缀”序列，以便输出。
- 需要进行全排列的元素集合，以便依次选做第一个元素。

这样可得到一个伪代码：

```
void print_permutation(序列 A, 集合 S)
{
    if(S 为空) 输出序列 A;
    else 按照从小到大的顺序依次考虑 S 的每个元素 v
    {
        print_permutation(在 A 的末尾填加 v 后得到的新序列, S-{v});
    }
}
```

暂时不用考虑序列 A 和集合 S 如何表示，首先理解一下上面的伪代码。递归边界是 S 为空的情形，这很好理解：现在序列 A 就是一个完整的排列，直接输出即可。接下来按照

从小到大的顺序考虑 S 中的每个元素，每次递归调用以 A 开头。

下面考虑程序实现。不难想到用数组表示序列 A，而集合 S 根本不用保存，因为它可以由序列 A 完全确定——A 中没有出现的元素都可以选。C 语言中的函数在接受数组参数时无法得知数组的元素个数，所以需要传一个已经填好的位置个数，或者当前需要确定的元素位置 cur，代码如下：

```
void print_permutation(int n, int* A, int cur) {
    if(cur == n) {                //递归边界
        for(int i = 0; i < n; i++) printf("%d ", A[i]);
        printf("\n");
    }
    else for(int i = 1; i <= n; i++) {    //尝试在A[cur]中填各种整数 i
        int ok = 1;
        for(int j = 0; j < cur; j++)
            if(A[j] == i) ok = 0;        //如果 i 已经在 A[0]~A[cur-1]出现过，则不能再选
        if(ok) {
            A[cur] = i;
            print_permutation(n, A, cur+1); //递归调用
        }
    }
}
```

循环变量 i 是当前考察的 A[cur]。为了检查元素 i 是否已经用过，上面的程序用到了一个标志变量 ok，初始值为 1（真），如果发现有某个 A[j] = i 时，则改为 0（假）。如果最终 ok 仍为 1，则说明 i 没有在序列中出现过，把它添加到序列末尾（A[cur] = i）后递归调用。

声明一个足够大的数组 A，然后调用 print_permutation(n, A, 0)，即可按字典序输出 1~n 的所有排列。

7.2.2 生成可重集的排列

如果把问题改成：输入数组 P，并按字典序输出数组 A 各元素的所有全排列，则需要对上述程序进行修改——把 P 加到 print_permutation 的参数列表中，然后把代码中的 if(A[j] == i) 和 A[cur] = i 分别改成 if(A[j] == P[i]) 和 A[cur] = P[i]。这样，只要把 P 的所有元素按从小到大的顺序排序，然后调用 print_permutation(n, P, A, 0) 即可。

这个方法看上去不错，可惜有一个小问题：输入 1 1 1 后，程序什么也不输出（正确答案应该是唯一的全排列 1 1 1），原因在于，这样禁止 A 数组中出现重复，而在 P 中本来就有重复元素时，这个“禁令”是错误的。

一个解决方法是统计 A[0]~A[cur-1] 中 P[i] 的出现次数 c1，以及 P 数组中 P[i] 的出现次数 c2。只要 c1 < c2，就能递归调用。

```
else for(int i = 0; i < n; i++) {
    int c1 = 0, c2 = 0;
```

```

for(int j = 0; j < cur; j++) if(A[j] == P[i]) c1++;
for(int j = 0; j < n; j++) if(P[i] == P[j]) c2++;
if(c1 < c2) {
    A[cur] = P[i];
    print_permutation(n, P, A, cur+1);
}
}

```

结果又如何呢？输入 1 1 1，输出了 27 个 1 1 1。遗漏没有了，但是出现了重复：先试着把第 1 个 1 作为开头，递归调用结束后再尝试用第 2 个 1 作为开头，递归调用结束后再尝试用第 3 个 1 作为开头，再一次递归调用。可实际上这 3 个 1 是相同的，应只递归 1 次，而不是 3 次。

换句话说，我们枚举的下标 i 应不重复、不遗漏地取遍所有 $P[i]$ 值。由于 P 数组已经排过序，所以只需检查 P 的第一个元素和所有“与前一个元素不相同”的元素，即只需在“for($i = 0; i < n; i++$)”和其后的花括号之前加上“if($i == 0 || P[i] != P[i-1]$)”即可。

至此，结果终于正确了。

7.2.3 解答树

假设 $n=4$ ，序列为 $\{1,2,3,4\}$ ，如图 7-1 所示的树显示出了递归函数的调用过程。其中，结点内部的序列表示 A ，位置 cur 用高亮表示，另外，由于从该处开始的元素和算法无关，因此用星号表示。

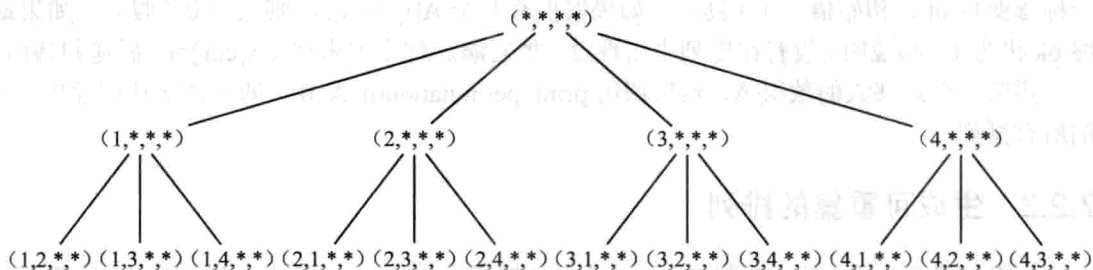


图 7-1 排列生成算法的解答树

这棵树和前面介绍过的二叉树不同。第 0 层（根）结点有 n 个子结点，第 1 层结点各有 $n-1$ 个子结点，第 2 层结点各有 $n-2$ 个子结点，第 3 层结点各有 $n-3$ 个子结点，……，第 n 层结点都没有子结点（即都是叶子），而每个叶子对应于一个排列，共有 $n!$ 个叶子。由于这棵树展示的是从“什么都没做”逐步生成完整解的过程，因此将其称为解答树。

提示 7-2：如果某问题的解可以由多个步骤得到，而每个步骤都有若干种选择（这些候选方案集可能会依赖于先前作出的选择），且可以用递归枚举法实现，则它的工作方式可以用解答树来描述。

这棵解答树一共有多少个结点呢？可以逐层查看：第 0 层有 1 个结点，第 1 层 n 个，第

2层有 $n*(n-1)$ 个结点（因为第1层的每个结点都有 $n-1$ 个结点），第3层有 $n*(n-1)*(n-2)$ 个（因为第2层的每个结点都有 $n-2$ 个结点），……，第 n 层有 $n*(n-1)*(n-2)*\dots*2*1=n!$ 个。

下面把它们加起来。为了推导方便，把 $n*(n-1)*(n-2)*\dots*(n-k)$ 写成 $n!/(n-k-1)!$ ，则所有结点之和为：

$$T(n) = \sum_{k=0}^{n-1} \frac{n!}{(n-k-1)!} = n! \sum_{k=0}^{n-1} \frac{1}{(n-k-1)!} = n! \sum_{k=0}^{n-1} \frac{1}{k!}$$

根据高等数学中的泰勒展开公式， $\lim_{n \rightarrow \infty} \sum_{k=0}^{n-1} \frac{1}{k!} = e$ ，因此 $T(n) < n! e = O(n!)$ 。由于叶子有 $n!$ 个，倒数第二层也有 $n!$ 个结点，因此上面的各层全部加起来也不到 $n!$ 。这是一个很重要的结论：在多数情况下，解答树上的结点几乎全部来源于最后一两层。和它们相比，上面的结点数可以忽略不计。

不熟悉泰勒展开公式也没有关系：可以写一个程序，输出 $\sum_{k=0}^{n-1} \frac{1}{k!}$ 随着 n 增大时的变化，并发现它能很快收敛。这就是计算机的优点之一——可以通过模拟避开数学推导。即使无法严密而精确地求解，也可以找到令人信服的实验数据。

7.2.4 下一个排列

枚举所有排列的另一个方法是从字典序最小排列开始，不停调用“求下一个排列”的过程。如何求下一个排列呢？C++的 STL 中提供了一个库函数 `next_permutation`。看看下面的代码片段，就会明白如何使用它了。

```
#include<cstdio>
#include<algorithm> //包含 next_permutation
using namespace std;
int main() {
    int n, p[10];
    scanf("%d", &n);
    for(int i = 0; i < n; i++) scanf("%d", &p[i]);
    sort(p, p+n); //排序，得到 p 的最小排列
    do {
        for(int i = 0; i < n; i++) printf("%d ", p[i]); //输出排列 p
        printf("\n");
    } while(next_permutation(p, p+n)); //求下一个排列
    return 0;
}
```

需要注意的是，上述代码同样适用于可重集。

提示 7-3：枚举排列的常见方法有两种：一是递归枚举，二是用 STL 中的 `next_permutation`。

7.3 子集生成

第7.2节中介绍了排列生成算法。本节介绍子集生成算法：给定一个集合，枚举所有可能的子集。为了简单起见，本节讨论的集合中没有重复元素。

7.3.1 增量构造法

第一种思路是一次选出一个元素放到集合中，程序如下：

```
void print_subset(int n, int* A, int cur) {
    for(int i = 0; i < cur; i++) printf("%d ", A[i]);    //打印当前集合
    printf("\n");
    int s = cur ? A[cur-1]+1 : 0;                        //确定当前元素的最小可能值
    for(int i = s; i < n; i++) {
        A[cur] = i;
        print_subset(n, A, cur+1);                      //递归构造子集
    }
}
```

和前面不同，由于 A 中的元素个数不确定，每次递归调用都要输出当前集合。另外，递归边界也不需要显式确定——如果无法继续添加元素，自然就不会再递归了。

上面的代码用到了定序的技巧：规定集合 A 中所有元素的编号从小到大排列，就不会把集合 $\{1, 2\}$ 按照 $\{1, 2\}$ 和 $\{2, 1\}$ 输出两次了。

提示 7-4：在枚举子集的增量法中，需要使用定序的技巧，避免同一个集合枚举两次。

这棵解答树上有 1024 个结点。这不难理解：每个可能的 A 都对应一个结点，而 n 元素集合恰好有 2^n 个子集， $2^{10}=1024$ 。

7.3.2 位向量法

第二种思路是构造一个位向量 $B[i]$ ，而不是直接构造子集 A 本身，其中 $B[i]=1$ ，当且仅当 i 在子集 A 中。递归实现如下：

```
void print_subset(int n, int* B, int cur) {
    if(cur == n) {
        for(int i = 0; i < cur; i++)
            if(B[i]) printf("%d ", i);                //打印当前集合
        printf("\n");
        return;
    }
    B[cur] = 1;                                         //选第 cur 个元素
```

```

print_subset(n, B, cur+1);
B[cur] = 0;                                //不选第 cur 个元素
print_subset(n, B, cur+1);
}

```

必须当“所有元素是否选择”全部确定完毕后才是一个完整的子集，因此仍然像以前那样当 $\text{if}(\text{cur} = n)$ 成立时才输出。现在的解答树上有 2047 个结点，比刚才的方法略多。这个也不难理解：所有部分解（不完整的解）也对应着解答树上的结点。

提示 7-5：在枚举子集的位向量法中，解答树的结点数略多，但在多数情况下仍然够快。

这是一棵 $n+1$ 层的二叉树（ cur 的范围从 $0 \sim n$ ），第 0 层有 1 个结点，第 1 层有 2 个结点，第 2 层有 4 个结点，第 3 层有 8 个结点，……，第 i 层有 2^i 个结点，总数为 $1+2+4+8+\dots+2^n=2^{n+1}-1$ ，和实验结果一致。如图 7-2 所示为这棵解答树。

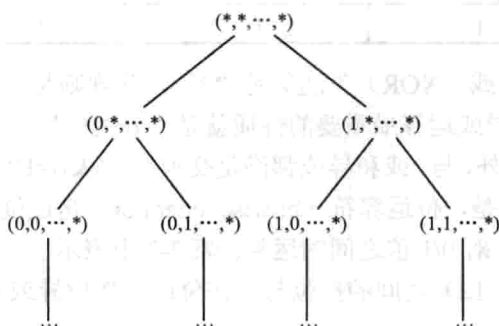


图 7-2 位向量法的解答树

这棵树依然符合前面的观察结果：最后几层结点数占整棵树的绝大多数。

7.3.3 二进制法

另外，还可以用二进制来表示 $\{0, 1, 2, \dots, n-1\}$ 的子集 S ：从右往左第 i 位（各位从 0 开始编号）表示元素 i 是否在集合 S 中。图 7-3 展示了二进制 0100011000110111 是如何表示集合 $\{0, 1, 2, 4, 5, 9, 10, 14\}$ 的。

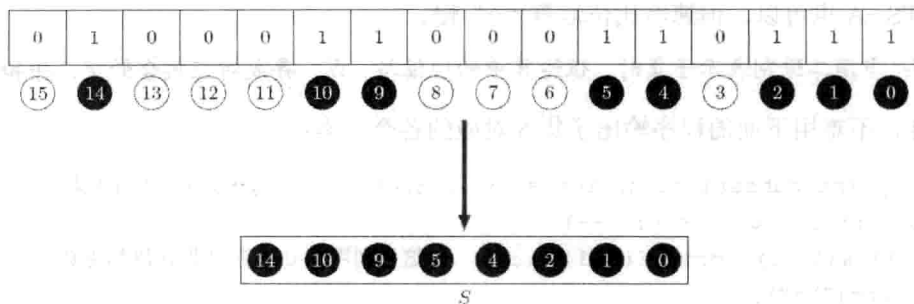


图 7-3 用二进制表示子集

注意：为了处理方便，最右边的位总是对应元素 0，而不是元素 1。

提示 7-6：可以用二进制表示子集，其中从右往左第 i 位（从 0 开始编号）表示元素 i 是否在集合中（1 表示“在”，0 表示“不在”）。

此时仅表示出集合是不够的，还需要对集合进行操作。幸运的是，常见的集合运算都可以用位运算符简单实现。最常见的二元位运算是与（&）、或（|）、非（!），它们和对应的逻辑运算非常相似，如表 7-1 所示。

表 7-1 C 语言中的二元位运算

A	B	A & B	A B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

表 7-1 中包括了“异或（XOR）”运算符“^”，其规则是“如果 A 和 B 不相同，则 $A \wedge B$ 为 1，否则为 0”。异或运算最重要的性质就是“开关性”——异或两次以后相当于没有异或，即 $A \wedge B \wedge B = A$ 。另外，与、或和异或都满足交换律： $A \& B = B \& A$ ， $A | B = B | A$ ， $A \wedge B = B \wedge A$ 。

与逻辑运算符不同的是，位运算符（bitwise operator）是逐位进行的——两个 32 位整数的“按位与”相当于 32 对 0/1 值之间的运算。表 7-2 中表示了二进制数 10110（十进制为 22）和 01100（十进制为 12）之间的按位与、按位或、按位异或的值，以及对应的集合运算的含义。

表 7-2 位运算与集合运算

	A	B	A & B	A B	A ^ B
二进制	10110	01100	00100	11110	11010
集合	{1,2,4}	{2,3}	{2}	{1,2,3,4}	{1,3,4}

不难看出， $A \& B$ 、 $A | B$ 和 $A \wedge B$ 分别对应集合的交、并和对称差。另外，空集为 0，全集 $\{0, 1, 2, \dots, n-1\}$ 的二进制为 n 个 1，即十进制的 $2^n - 1$ 。为了方便，往往在程序中把全集定义为 $\text{ALL_BITS} = (1 \ll n) - 1$ ，则 A 的补集就是 $\text{ALL_BITS} \wedge A$ 。当然，直接用整数减法 $\text{ALL_BITS} - A$ 也可以，但速度比位运算“^”慢。

提示 7-7：当用二进制表示子集时，位运算中的按位与、或、异或对应集合的交、并和对称差。

这样，不难用下面的程序输出子集 S 对应的各个元素：

```
void print_subset(int n, int s) { //打印{0, 1, 2, ..., n-1}的子集 S
    for(int i = 0; i < n; i++)
        if(s & (1 << i)) printf("%d ", i); //这里利用了 C 语言“非 0 值都为真”的规定
    printf("\n");
}
```

而枚举子集和枚举整数一样简单：

```
for(int i = 0; i < (1<<n); i++)    //枚举各子集所对应的编码 0, 1, 2, ..., 2^n-1
    print_subset(n, i);
```

提示 7-8：从代码量看，枚举子集的最简单方法是二进制法。

7.4 回溯法

无论是排列生成还是子集枚举，前面都给出了两种思路：递归构造和直接枚举。直接枚举法的优点是思路和程序都很简单，缺点在于无法简便地减小枚举量——必须生成（generate）所有可能的解，然后一一检查（test）。

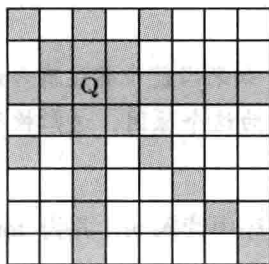
另一方面，在递归构造中，生成和检查过程可以有机结合起来，从而减少不必要的枚举。这就是本节的主题——回溯法（backtracking）。

回溯法的应用范围很广，只要能把待求解的问题分成不太多的步骤，每个步骤又只有不太多的选择，都可以考虑应用回溯法。为什么说“不太多”呢？想象一棵包含 L 层，每层的分支因子均为 b 的解答树，其结点数高达 $1+b+b^2+\dots+b^{L-1}=\frac{b^L-1}{b-1}$ 。无论是 b 太大还是 L 太大，结点数都会是一个天文数字。

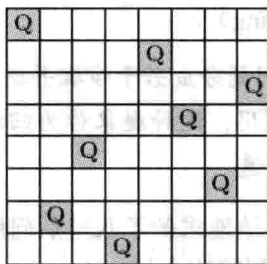
回溯法是初学者学习暴力法的第一个障碍，学习时间短则数天，长则数月甚至一年以上。为了减少不必要的困扰，在学习回溯法之前，请读者确保 7.2 节和 7.3 节的所有递归程序都可以熟练、准确地写出。

7.4.1 八皇后问题

在棋盘上放置 8 个皇后，使得它们互不攻击，此时每个皇后的攻击范围为同行同列和同对角线，要求找出所有解，如图 7-4 所示。



(a) 皇后的攻击范围



(b) 一个可行解

图 7-4 八皇后问题

【分析】

最简单的思路是把问题转化为“从 64 个格子中选一个子集”，使得“子集中恰好有 8

个格子，且任意两个选出的格子都不在同一行、同一列或同一个对角线上”。这正是子集枚举问题。然而，64 个格子的子集有 2^{64} 个，太大了，这并不是一个很好的模型。

第二个思路是把问题转化为“从 64 个格子中选 8 个格子”，这是组合生成问题。根据组合数学，有 $C_{64}^8 = 4.426 \times 10^9$ 种方案，比第一种方案优秀，但仍然不够好。

经过思考，不难发现以下事实：恰好每行每列各放置一个皇后。如果用 $C[x]$ 表示第 x 行皇后的列编号，则问题变成了全排列生成问题。而 0~7 的排列一共只有 $8! = 40320$ 个，枚举量不会超过它。

提示 7-9：在编写递归枚举程序之前，需要深入分析问题，对模型精雕细琢。一般还应解答树的结点数有一个粗略的估计，作为评价模型的重要依据，如图 7-5 所示。

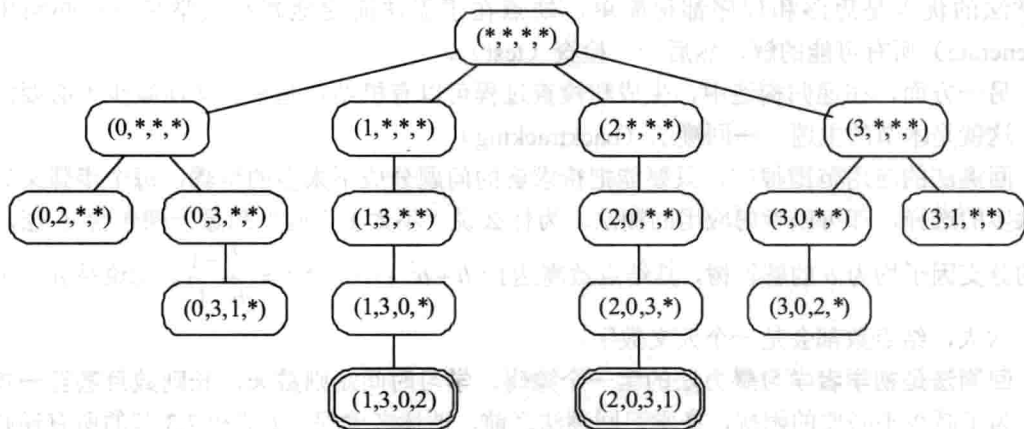


图 7-5 四皇后问题的解答树

图 7-5 中给出了四皇后问题的完整解答树。它只有 17 个结点，比 $4! = 24$ 小。为什么会这样呢？这是因为有些结点无法继续扩展。例如，在 $(0,2,*)$ 中，第 2 行无论将皇后放到哪里，都会和第 0 行和第 1 行中已放好的皇后发生冲突，其他还未放置的皇后更是如此。

在这种情况下，递归函数将不再递归调用它自身，而是返回上一层调用，这种现象称为回溯（backtracking）。

提示 7-10：当把问题分成若干步骤并递归求解时，如果当前步骤没有合法选择，则函数将返回上一级递归调用，这种现象称为回溯。正是因为这个原因，递归枚举算法常被称为回溯法，应用十分普遍。

下面的程序简洁地求解了八皇后问题。在主程序中读入 n ，并为 tot 清零，然后调用 $search(0)$ ，即可得到解的个数 tot 。

```

void search(int cur) {
    if(cur == n) tot++;           //递归边界。只要走到了这里，所有皇后必然不冲突
    else for(int i = 0; i < n; i++) {
        int ok = 1;

```

```

C[cur] = i; //尝试把第 cur 行的皇后放在第 i 列
for(int j = 0; j < cur; j++) //检查是否和前面的皇后冲突
    if(C[cur] == C[j] || cur-C[cur] == j-C[j] || cur+C[cur] == j+C[j])
        { ok = 0; break; }
if(ok) search(cur+1); //如果合法，则继续递归
}
}

```

注意：既然是逐行放置的，则皇后肯定不会横向攻击，因此只需检查是否纵向和斜向攻击即可。条件“ $cur-C[cur] == j-C[j] \parallel cur+C[cur] == j+C[j]$ ”用来判断皇后 $(cur, C[cur])$ 和 $(j, C[j])$ 是否在同一条对角线上。其原理可以用图 7-6 来说明。

0	1	2	3	4	5	6	7
-1	0	1	2	3	4	5	6
-2	-1	0	1	2	3	4	5
-3	-2	-1	0	1	2	3	4
-4	-3	-2	-1	0	1	2	3
-5	-4	-3	-2	-1	0	1	2
-6	-5	-4	-3	-2	-1	0	1
-7	-6	-5	-4	-3	-2	-1	0

(a) 格子 (x,y) 的 $y-x$ 值标识了主对角线

0	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	12
6	7	8	9	10	11	12	13
7	8	9	10	11	12	13	14

(b) 格子 (x,y) 的 $x+y$ 值标识了副对角线

图 7-6 棋盘中的对角线标识

结点数似乎很难进一步减少了，但程序效率可以继续提高：利用二维数组 `vis[2][n]` 直接判断当前尝试的皇后所在的列和两个对角线是否已有其他皇后。注意到主对角线标识 $y-x$ 可能为负，存取时要加上 n 。

```

void search(int cur) {
    if(cur == n) tot++;
    else for(int i = 0; i < n; i++) {
        if(!vis[0][i] && !vis[1][cur+i] && !vis[2][cur-i+n]) {
            //利用二维数组直接判断
            C[cur] = i; //如果不用打印解，整个 C 数组都可以省略
            vis[0][i] = vis[1][cur+i] = vis[2][cur-i+n] = 1; //修改全局变量
            search(cur+1);
            vis[0][i] = vis[1][cur+i] = vis[2][cur-i+n] = 0; //切记！一定要改回来
        }
    }
}

```

上面的程序有个极其关键的地方：`vis` 数组的使用。`vis` 数组的确切含义是什么？它表示已经放置的皇后占据了哪些列、主对角线和副对角线。将来放置的皇后不应该修改这些值——至少“看上去没有修改”。一般地，如果在回溯法中修改了辅助的全局变量，则一

定要及时把它们恢复原状（除非故意保留所做修改）。若不信，可以把“`vis[0][i]=vis[1][cur+i]=vis[2][cur-i+n]=0`”注释掉，验证还能否正确求解八皇后问题。另外，在调用之前一定要把 `vis` 数组清空。

提示 7-11：如果在回溯法中使用了辅助的全局变量，则一定要及时把它们恢复原状。特别地，若函数有多个出口，则需在每个出口处恢复被修改的值。

7.4.2 其他应用举例

例题 7-4 素数环（Prime Ring Problem, UVa 524）

输入正整数 n ，把整数 $1, 2, 3, \dots, n$ 组成一个环，使得相邻两个整数之和均为素数。输出时从整数 1 开始逆时针排列。同一个环应恰好输出一次。 $n \leq 16$ 。

样例输入：

6

样例输出：

1 4 3 2 5 6

1 6 5 2 3 4

【分析】

由模型不难得到：每个环对应于 $1 \sim n$ 的一个排列，但排列总数高达 $16! = 2 \times 10^{13}$ ，生成-测试法会超时吗？下面进行实验：

```
for(int i = 2; i <= n*2; i++) isp[i] = is_prime(i); //生成素数表，加快后续判断
for(int i = 0; i < n; i++) A[i] = i+1; //第一个排列
do {
    int ok = 1;
    for(int i = 0; i < n; i++) if(!isp[A[i]+A[(i+1)%n]]) { ok = 0; break; } //判断合法性
    if(ok){
        for(int i = 0; i < n; i++) printf("%d ", A[i]); //输出序列
        printf("\n");
    }
}while(next_permutation(A+1, A+n)); //1 的位置不变
```

运行后发现，当 $n=12$ 时就已经很慢，而当 $n=16$ 时无法运行出结果。下面试试回溯法：

```
void dfs(int cur){
    if(cur == n && isp[A[0]+A[n-1]]){ //递归边界。别忘了测试第一个数和最后一个数
        for(int i = 0; i < n; i++) printf("%d ", A[i]); //打印方案
        printf("\n");
    }
}
```

```

else for(int i = 2; i <= n; i++) //尝试放置每个数 i
{
    if(!vis[i] && isp[i+A[cur-1]]){ //如果 i 没有用过, 并且与前一个数之和为素数
        A[cur] = i;
        vis[i] = 1; //设置使用标志
        dfs(cur+1);
        vis[i] = 0; //清除标志
    }
}
}

```

回溯法比生成-测试法快了很多, 即使 $n=18$ 速度也不错。将上面的函数名设为 `dfs` 并不是巧合——从解答树的角度讲, 回溯法正是按照深度优先的顺序在遍历解答树。在后面的内容中, 还将学习更多遍历解答树的方法。

提示 7-12: 如果最坏情况下的枚举量很大, 应该使用回溯法而不是生成-测试法。

例题 7-5 困难的串 (Krypton Factor, UVa 129)

如果一个字符串包含两个相邻的重复子串, 则称它是“容易的串”, 其他串称为“困难的串”。例如, BB、ABCDACABCAB、ABCDABCD 都是容易的串, 而 D、DC、ABDAB、CBABCBA 都是困难的串。

输入正整数 n 和 L , 输出由前 L 个字符组成的、字典序第 k 小的困难的串。例如, 当 $L=3$ 时, 前 7 个困难的串分别为 A、AB、ABA、ABAC、ABACA、ABACAB、ABACABA。输入保证答案不超过 80 个字符。

样例输入:

```

7 3
30 3

```

样例输出:

```

ABACABA
ABACABCACBABCABACABCACBACABA

```

【分析】

基本框架不难确定: 从左到右依次考虑每个位置上的字符。因此, 问题的关键在于如何判断当前字符串是否已经存在连续的重复子串。例如, 如何判断 ABACABA 是否包含连续重复子串呢? 一种方法是检查所有长度为偶数的子串, 分别判断每个子串的前一半是否等于后一半。尽管是正确的, 但这个方法做了很多无用功。还记得八皇后问题中是怎么判断合法性的吗? 判断当前皇后是否和前面的皇后冲突, 但并不判断以前的皇后是否相互冲突——那些皇后在以前已经判断过了。同样的道理, 我们只需要判断当前串的后缀, 而非所有子串。

提示 7-13: 在回溯法中, 应注意避免不必要的判断, 就像在八皇后问题中那样, 只需判断新皇后和之前的皇后是否冲突, 而不必判断以前的皇后是否相互冲突。

程序如下：

```
int dfs(int cur){                                     //返回 0 表示已经得到解，无须继续搜索
    if(cnt++ == n){
        for(int i = 0; i < cur; i++) printf("%c", 'A'+S[i]); //输出方案
        printf("\n");
        return 0;
    }
    for(int i = 0; i < L; i++){
        S[cur] = i;
        int ok = 1;
        for(int j = 1; j*2 <= cur+1; j++){ //尝试长度为 j*2 的后缀
            int equal = 1;
            for(int k = 0; k < j; k++) //检查后半是否等于前半
                if(S[cur-k] != S[cur-k-j]) { equal = 0; break; }
            if(equal) { ok = 0; break; } //后半等于前半，方案不合法
        }
        if(ok) if(!dfs(cur+1)) return 0; //递归搜索。如果已经找到解，则直接退出
    }
    return 1;
}
```

有意思的是， $L=2$ 时一共只有 6 个串；当 $L \geq 3$ 时就很少回溯了。事实上，当 $L=3$ 时，可以构造出无限长的串，不存在相邻重复子串。

例题 7-6 带宽 (Bandwidth, UVa 140)

给出一个 n ($n \leq 8$) 个结点的图 G 和一个结点的排列，定义结点 i 的带宽 $b(i)$ 为 i 和相邻结点在排列中的最远距离，而所有 $b(i)$ 的最大值就是整个图的带宽。给定图 G ，求出让带宽最小的结点排列，如图 7-7 所示。

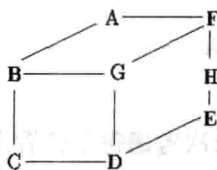


图 7-7 图 G

下面两个排列的带宽分别为 6 和 5。具体来说，图 7-8 (a) 中各个结点的带宽分别为 6, 6, 1, 4, 1, 1, 6, 6，图 7-8 (b) 中各个结点的带宽分别为 5, 3, 1, 4, 3, 5, 1, 4。

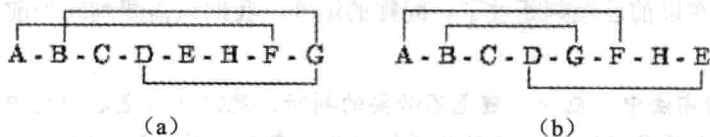


图 7-8 两个排列的带宽

【分析】

如果不考虑效率，本题可以递归枚举全排列，分别计算带宽，然后选取最小的一种方案。能否优化呢？和八皇后问题不同的是：八皇后问题有很多可行性约束（feasibility constraint），可以在得到完整解之前避免扩展那些不可行的结点，但本题并没有可行性约束——任何排列都是合法的。难道只能扩展所有结点吗？当然不是。

可以记录下目前已经找到的最小带宽 k 。如果发现已经有某两个结点的距离大于或等于 k ，再怎么扩展也不可能比当前解更优，应当强制把它“剪”掉，就像园丁在花园里为树修剪枝叶一样，也可以为解答树“剪枝（prune）”。

除此之外，还可以剪掉更多的枝叶。如果在搜索到结点 u 时， u 结点还有 m 个相邻点没有确定位置，那么对于结点 u 来说，最理想的情况就是这 m 个结点紧跟在 u 后面，这样的结点带宽为 m ，而其他任何“非理想情况”的带宽至少为 $m+1$ 。这样，如果 $m \geq k$ ，即“在最理想的情况下都不能得到比当前最优解更好的方案”，则应当剪枝。

提示 7-14：在求最优解的问题中，应尽量考虑最优性剪枝。这往往需要记录下当前最优解，并且想办法“预测”一下从当前结点出发是否可以扩展到更好的方案。具体来说，先计算一下最理想情况可以得到怎样的解，如果连理想情况都无法得到比当前最优解更好的方案，则剪枝。

例题 7-7 天平难题（Mobile Computing, ACM/ICPC Tokyo 2005, UVa1354）

给出房间的宽度 r 和 s 个挂坠的重量 w_i 。设计一个尽量宽（但宽度不能超过房间宽度 r ）的天平，挂着所有挂坠。

天平由一些长度为 1 的木棍组成。木棍的每一端要么挂一个挂坠，要么挂另外一个木棍。如图 7-9 所示，设 n 和 m 分别是两端挂的总重量，要让天平平衡，必须满足 $n \cdot a = m \cdot b$ 。

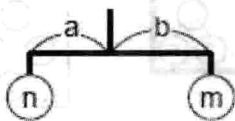
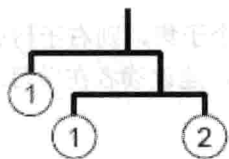
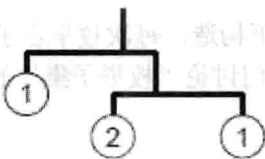


图 7-9 天平

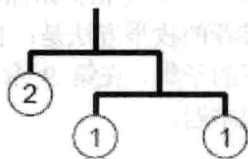
例如，如果有 3 个重量分别为 1, 1, 2 的挂坠，有 3 种平衡的天平，如图 7-10 所示。



$$\text{width} = 1 + (1/3)$$



$$\text{width} = 1 + (2/3)$$



$$\text{width} = 1 + (1/2)$$

图 7-10 3 种平衡的天平

挂坠的宽度忽略不计，且不同的子天平可以相互重叠。如图 7-11 所示，宽度为 $(1/3)+1+(1/4)$ 。

输入第一行为数据组数。每组数据前两行为房间宽度 r 和挂坠数目 s ($0 < r < 10$, $1 \leq s \leq$

6)。以下 s 行每行为一个挂坠的重量 w_i ($1 \leq w_i \leq 1000$)。输入保证不存在天平的宽度恰好在 $r-10^{-5}$ 和 $r+10^{-5}$ 之间（这样可以保证不会出现精度问题）。对于每组数据，输出最优天平的宽度。如果无解，输出-1。你的输出和标准答案的绝对误差不应超过 10^{-8} 。

【分析】

如果把挂坠和木棍都作为结点，则一个天平对应一棵二叉树，如题目中给出的，挂坠为 1, 1, 2 的 3 个天平如图 7-12 所示。

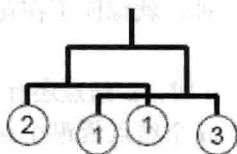


图 7-11 子天平相互重叠

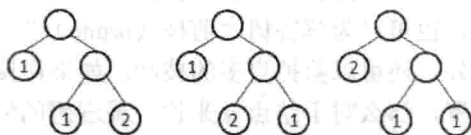


图 7-12 与天平对应的二叉树

对于一棵确定二叉树，可以计算出每个挂坠的确切位置，进而计算出整个天平的宽度，所以本题的核心任务是：枚举二叉树。

如何枚举二叉树呢？最直观的方法是沿用回溯法框架，每次选择两个结点组成一棵子树，递归 $s-1$ 层即可。以 4 个挂坠 1, 1, 2, 3 为例，下面是解答树的一部分（每个结点的子树并没有全部画出），如图 7-13 所示。

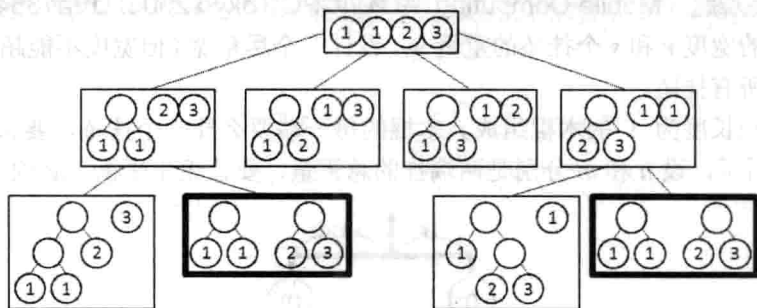


图 7-13 解答树

上面的方法已经足够解决本题，但还有优化的余地，因为有些二叉树被枚举了多次（如图 7-13 中的两个粗框结点）。

推荐的枚举方法是：自顶向下构造，每次枚举左子树用到哪个子集，则右子树就是使用剩下的子集。在第 9 章中会专门讨论“枚举子集”的高效算法，建议读者在学习之后重新实现本题。

7.5 路径寻找问题

在第 6 章中曾经介绍过图的遍历。很多问题都可以归结为图的遍历，但这些问题中的图却不是事先给定、从程序读入的，而是由程序动态生成的，称为**隐式图**。本节和前面介

绍的回溯法不同：回溯法一般是要找到一个（或者所有）满足约束的解（或者某种意义上下的最优解），而状态空间搜索一般是要找到一个从初始状态到终止状态的路径。

提示 7-15：路径寻找问题可以归结为隐式图的遍历，它的任务是找到一条从初始状态到终止状态的最优路径，而不是像回溯法那样找到一个符合某些要求的解。

八数码问题。编号为 1~8 的 8 个正方形滑块被摆成 3 行 3 列（有一个格子留空），如图 7-14 所示。每次可以把与空格相邻的滑块（有公共边才算相邻）移到空格中，而它原来的位置就成为了新的空格。给定初始局面和目标局面（用 0 表示空格），你的任务是计算出最少的移动步数。如果无法到达目标局面，则输出 -1。

2	6	4
1	3	7
	5	8

8	1	5
7	3	6
4		2

图 7-14 八数码问题举例

样例输入：

```
2 6 4 1 3 7 0 5 8
8 1 5 7 3 6 4 0 2
```

样例输出：

```
31
```

【分析】

不难把八数码问题归结为图上的最短路问题，图的“结点”就是 9 个格子中的滑块编号（从上到下、从左到右把它们放到一个包含 9 个元素的数组中）。根据第 6 章的讲解，无权图上的最短路问题可以用 BFS 求解，代码如下：

```
typedef int State[9];           //定义“状态”类型
const int maxstate = 1000000;
State st[maxstate], goal;      //状态数组。所有状态都保存在这里
int dist[maxstate];           //距离数组
//如果需要打印方案，可以在这里加一个“父亲编号”数组 int fa[maxstate]

const int dx[] = {-1, 1, 0, 0};
const int dy[] = {0, 0, -1, 1};
//BFS，返回目标状态在 st 数组下标
int bfs() {
    init_lookup_table();       //初始化查找表
    int front = 1, rear = 2;   //不使用下标 0，因为 0 被看作“不存在”
    while(front < rear) {
        State& s = st[front];  //用“引用”简化代码
```

```

    if(memcmp(goal, s, sizeof(s)) == 0) return front; //找到目标状态, 成功返回
    int z;
    for(z = 0; z < 9; z++) if(!s[z]) break;           //找“0”的位置
    int x = z/3, y = z%3;                             //获取行列编号 (0~2)
    for(int d = 0; d < 4; d++) {
        int newx = x + dx[d];
        int newy = y + dy[d];
        int newz = newx * 3 + newy;
        if(newx >= 0 && newx < 3 && newy >= 0 && newy < 3){ //如果移动合法
            State& t = st[rear];
            memcpy(&t, &s, sizeof(s));                //扩展新结点
            t[newz] = s[z];
            t[z] = s[newz];
            dist[rear] = dist[front] + 1;              //更新新结点的距离值
            if(try_to_insert(rear)) rear++;            //如果成功插入查找表, 修改队尾指针
        }
    }
    front++;                                           //扩展完毕后再修改队首指针
}
return 0;                                           //失败
}

```

注意, 此处用到了 `cstring` 中的 `memcmp` 和 `memcpy` 完成整块内存的比较和复制, 比用循环比较和循环赋值要快。主程序很容易实现:

```

int main(){
    for(int i = 0; i < 9; i++) scanf("%d", &st[1][i]); //起始状态
    for(int i = 0; i < 9; i++) scanf("%d", &goal[i]); //目标状态
    int ans = bfs(); //返回目标状态的下标
    if(ans > 0) printf("%d\n", dist[ans]);
    else printf("-1\n");
    return 0;
}

```

注意, 应在调用 `bfs` 函数之前设置好 `st[1]` 和 `goal`。上面的代码几乎是完整的, 唯一没有涉及的是 `init_lookup_table()` 和 `try_to_insert(rear)` 的实现。为什么会有这两项呢? 还记得 BFS 中的“判重”操作吗? 在 DFS 中可以检查 `idx` 来判断结点是否已经访问过; 在求最短路的 BFS 中用 `d` 值是否为 -1 来判断结点是否访问过, 不管用哪种方法, 作用是相同的: 避免同一个结点访问多次。树的 BFS 不需要判重, 因为根本不会重复; 但对于图来说, 如果不判重, 时间和空间都将产生极大的浪费。

如何判重呢? 难道要声明一个 9 维数组 `vis`, 然后执行 `if(vis[s[0]][s[1]][s[2]]...s[8])`? 无论程序好不好看, 9 维数组的每维都要包含 9 个元素, 一共有 $9^9=387420489$ 项, 太多了, 数组开不下。实际的结点数并没有这么多 (0~8 的排列总共只有 $9!=362880$ 个), 为什么 9

维数组开不下呢？原因在于，这样的用法存在大量的浪费——数组中有很多项都没有被用到，但却占据了空间。

下面通过讨论3种常见的方法来解决这个问题，同时将它们用到八数码问题中。

第1种方法是：把排列“变成”整数，然后只开一个一维数组。也就是说，设计一套排列的编码（encoding）和解码（decoding）函数，把0~8的全排列和0~362879的整数一一对应起来。第10章中将详细讨论编码和解码问题，这里先给出代码以便读者形成一个感性认识：

```
int vis[362880], fact[9];
void init_lookup_table(){
    fact[0] = 1;
    for(int i = 1; i < 9; i++) fact[i] = fact[i-1] * i;
}
int try_to_insert(int s){
    int code = 0; //把st[s]映射到整数 code
    for(int i = 0; i < 9; i++){
        int cnt = 0;
        for(int j = i+1; j < 9; j++) if(st[s][j] < st[s][i]) cnt++;
        code += fact[8-i] * cnt;
    }
    if(vis[code]) return 0;
    return vis[code] = 1;
}
```

尽管原理巧妙，时间效率也非常高，但编码解码法的适用范围并不大：如果隐式图的总结点数非常大，编码也将会很大，数组还是开不下。

第2种方法是使用哈希（hash）技术。简单地说，就是要把结点“变成”整数，但不必是一一对应。换句话说，只需要设计一个所谓的哈希函数 $h(x)$ ，然后将任意结点 x 映射到某个给定范围 $[0, M-1]$ 的整数即可，其中 M 是程序员根据可用内存大小自选的。在理想情况下，只需开一个大小为 M 的数组就能完成判重，但此时往往会有不同结点的哈希值相同，因此需要把哈希值相同的状态组织成链表，细节参见下面的代码：

```
const int hashsize = 1000003;
int head[hashsize], next[maxstate];
void init_lookup_table() { memset(head, 0, sizeof(head)); }
int hash(State& s){
    int v = 0;
    for(int i = 0; i < 9; i++) v = v * 10 + s[i]; //把9个数字组合成9位数
    return v % hashsize; //确保 hash 函数值是不超过 hash 表的大小的非负整数
}
int try_to_insert(int s){
    int h = hash(st[s]);
```

```

int u = head[h]; //从表头开始查找链表
while(u){
    if(memcmp(st[u],st[s], sizeof(st[s]))==0) return 0; //找到了，插入失败
    u = next[u]; //顺着链表继续找
}
next[s] = head[h]; //插入到链表中
head[h] = s;
return 1;
}

```

哈希表的执行效率高，适用范围也很广。除了 BFS 中的结点判重外，还可以用到其他需要快速查找的地方。不过需要注意的是：在哈希表中，对效率起到关键作用的是哈希函数。如果哈希函数选取得当，几乎不会有结点的哈希值相同，且此时链表查找的速度也较快；但如果冲突严重，整个哈希表会退化成少数几条长长的链表，查找速度将非常缓慢。有趣的是，前面的编码函数可以看作是一个完美的哈希函数，不需要解决冲突。不过，如果事先并不知道它是完美的，也就不敢像前面一样只开一个 vis 数组。哈希技术还有很多值得探讨的地方，建议读者在网上查找相关资料。

第3种方法是用 STL 集合 t。把状态转化成 9 位十进制整数，就可以用 set<int>判重了：

```

set<int> vis;
void init_lookup_table() { vis.clear(); }
int try_to_insert(int s){
    int v = 0;
    for(int i = 0; i < 9; i++) v = v * 10 + st[s][i];
    if(vis.count(v)) return 0;
    vis.insert(v);
    return 1;
}

```

在刚才的 3 种实现中，使用 STL 集合的代码最简单，但时间效率也最低（若此时不用 -O2 优化则速度劣势更加明显）。建议读者在时间紧迫或对效率要求不太高的情况下使用，或者仅把它作为“跳板”——先写一个 STL 版的程序，确保主算法正确，然后把 set 替换成自己写的哈希表。

提示 7-16：隐式图遍历需要用一个结点查找表来判重。一般来说，使用 STL 集合实现的代码最简单，但效率也较低。如果题目对时间要求很高，可以先把 STL 集合版的程序调试通过，然后转化为哈希表甚至完美哈希表。

某些特定的 STL 实现中还有 hash_set，它正是基于前面的哈希表，但它并不是标准 C++ 的一部分，因此不是所有情况下都可用。

例题 7-8 倒水问题 (Fill, UVa 10603)

有装满水的 6 升的杯子、空的 3 升杯子和 1 升杯子，3 个杯子中都没有刻度。在不使用其他道具的情况下，是否可以量出 4 升的水呢？

方法如图 7-15 所示。

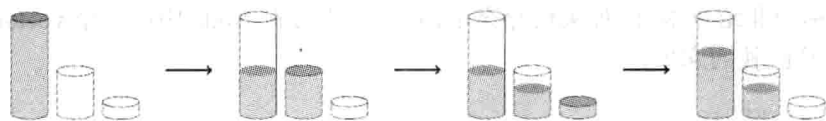


图 7-15 倒水问题：一种方法是 $(6,0,0) \rightarrow (3,3,0) \rightarrow (3,2,1) \rightarrow (4,2,0)$

注意：由于没有刻度，用杯子 x 给杯子 y 倒水时必须一直持续到把杯子 y 倒满或者把杯子 x 倒空，而不能中途停止。

你的任务是解决一般性的问题：设 3 个杯子的容量分别为 a, b, c ，最初只有第 3 个杯子装满了 c 升水，其他两个杯子为空。最少需要倒多少升水才能让某一个杯子中的水有 d 升呢？如果无法做到恰好 d 升，就让某一个杯子里的水是 d' 升，其中 $d' < d$ 并且尽量接近 d 。

($1 \leq a, b, c, d \leq 200$)。要求输出最少的倒水量和目标水量 (d 或者 d')。

【分析】

假设在某一时刻，第 1 个杯子中有 v_0 升水，第 2 个杯子中有 v_1 升水，第 3 个杯子中有 v_2 升水，称当时的系统状态为 (v_0, v_1, v_2) 。这里再次提到了“状态”这个词，它是理解很多概念和算法的关键。简单地说，它就是“对系统当前状况的描述”。例如，在国际象棋中，当前游戏者和棋盘上的局面就是刻画游戏进程的状态。

把“状态”想象成图中的结点，可以得到如图 7-16 所示的状态图 (state graph)。

由于无论如何倒，杯子中的水量都是整数（按照倒水次数归纳即可），因此第 3 个杯子的水量最多只有 $0, 1, 2, \dots, c$ 共 $c+1$ 种可能；同理，第 2 个杯子的水量一共只有 $b+1$ 种可能，第 1 个杯子一共只有 $a+1$ 种可能，因此理论上状态最多有 $(a+1)(b+1)(c+1) = 8120601$ 种可能性，有点大。幸运的是，上面的估计是不精确的。由于水的总量 x 永远不变，如果有两个状态的前两个杯子的水量都相同，则第 3 个杯子的水量也相同。换句话说，最多可能的状态数不会超过 $201^2 = 40401$ 。

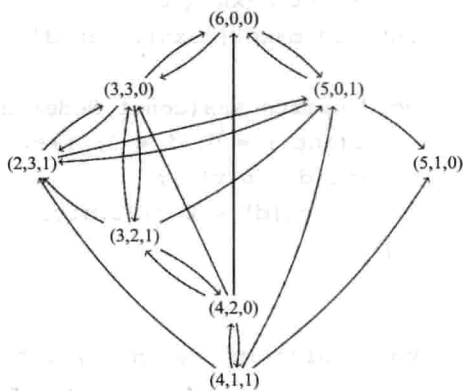


图 7-16 倒水问题的状态图

注意：本题的目标是倒的水量最少，而不是步数最少。实际上，水量最少时步数不一定最少，例如 $a=1, b=12, c=15, d=7$ ，倒水量最少的方案是 $C \rightarrow A, A \rightarrow B$ 重复 7 次，最后 C 里有 7 升水。一共 14 步，总水量也是 14。还有一种方法是 $C \rightarrow B$ ，然后 $B \rightarrow A, A \rightarrow C$ 重复 4 次，最后 C 里有 7 升水。一共只有 10 步，但总水量多达 20。

因此，需要改一下算法：不是每次取出步数最少的结点进行扩展，而是取出水量最少的结点进行扩展。这样的程序只需要把队列 `queue` 换成优先队列 `priority_queue`，其他部分的代码不变。下面的代码把状态（三元组）和 `dist` 合起来定义为了一个 `Node` 类型，是一种常见的写法。如果要打印路径，需要把访问过的所有结点放在一个 `nodes` 数组中，然后在

Node 中加一个变量 fa, 表示父结点在 nodes 数组中的下标, 而在队列中只存结点在 nodes 数组中的下标而非结点本身。如果内存充足, 也可以直接在 Node 中用一个 vector 保存路径, 省去顺着 fa 往回找的麻烦。

```
#include<cstdio>
#include<cstring>
#include<queue>
using namespace std;

struct Node {
    int v[3], dist;
    bool operator < (const Node& rhs) const {
        return dist > rhs.dist;
    }
};

const int maxn = 200 + 5;
int vis[maxn][maxn], cap[3], ans[maxn];

void update_ans(const Node& u) {
    for(int i = 0; i < 3; i++) {
        int d = u.v[i];
        if(ans[d] < 0 || u.dist < ans[d]) ans[d] = u.dist;
    }
}

void solve(int a, int b, int c, int d) {
    cap[0] = a; cap[1] = b; cap[2] = c;
    memset(vis, 0, sizeof(vis));
    memset(ans, -1, sizeof(ans));
    priority_queue<Node> q;

    Node start;
    start.dist = 0;
    start.v[0] = 0; start.v[1] = 0; start.v[2] = c;
    q.push(start);

    vis[0][0] = 1;
    while(!q.empty()) {
        Node u = q.top(); q.pop();
        update_ans(u);
        if(ans[d] >= 0) break;
        for(int i = 0; i < 3; i++)
            for(int j = 0; j < 3; j++) if(i != j) {
```

```

    if(u.v[i] == 0 || u.v[j] == cap[j]) continue;
    int amount = min(cap[j], u.v[i] + u.v[j]) - u.v[j];
    Node u2;
    memcpy(&u2, &u, sizeof(u));
    u2.dist = u.dist + amount;
    u2.v[i] -= amount;
    u2.v[j] += amount;
    if(!vis[u2.v[0]][u2.v[1]]) {
        vis[u2.v[0]][u2.v[1]] = 1;
        q.push(u2);
    }
}
while(d >= 0) {
    if(ans[d] >= 0) {
        printf("%d %d\n", ans[d], d);
        return;
    }
    d--;
}

int main() {
    int T, a, b, c, d;
    scanf("%d", &T);
    while(T--) {
        scanf("%d%d%d%d", &a, &b, &c, &d);
        solve(a, b, c, d);
    }
    return 0;
}

```

需要注意的是：上述算法非常直观，正确性却不是显然的。事实上，笔者目前没有找到反例，但也无法严格证明它是正确的^①。幸运的是，上述算法稍加修改，就可以得到第11章中要介绍的 Dijkstra 算法，从而保证算法的正确性。等学完 Dijkstra 算法之后，读者不妨回来再看看这道题目，相信会有新的体会。希望读者能够通过这个例题看到搜索和图论这两个看似无关的主题之间的联系。

例题 7-9 万圣节后的早晨 (The Morning after Halloween, Japan 2007, UVa1601)

$w * h$ ($w, h \leq 16$) 网格上有 n ($n \leq 3$) 个小写字母 (代表鬼)。要求把它们分别移动到大写字母里。每步可以有多个鬼同时移动 (均为往上下左右 4 个方向之一移动)，但每

^① 如果有读者找到反例或者正确性证明，请联系笔者或者出版社，我们会在重印时更正。

步结束之后任何两个鬼不能占用同一个位置，也不能在一步之内交换位置。例如如图 7-17 所示的局面：一共有 4 种移动方式，如图 7-18 所示。

```
####
ab#
#c##
####
```

图 7-17 题设局面

```
####  ####  ####  ####
ab#  a b#  acb#  ab #
#c##  #c##  # ##  #c##
####  ####  ####  ####
```

图 7-18 4 种移动方式

输入保证所有空格连通，所有障碍格也连通，且任何一个 2×2 子网格中至少有一个障碍格。输出最少的步数。输入保证有解。

【分析】

以当前 3 个小写字母的位置为状态，则问题转化为图上的最短路问题。状态总数为 256^3 ，每次转移时需要 5^3 枚举每一个小写字母下一步的走法（上下左右加上“不动”）。可惜状态数已经很大了，转移代价又比较高，很容易超时，需要优化。

首先是优化转移代价。条件“任何一个 2×2 子网格中至少有一个障碍格”暗示着很多格子都是障碍，并且大部分空地都和障碍相邻，因此不是所有 4 个方向都能移动，因此可以把所有空格提出来建立一张图，而不是每次临时判断 5 种方案是否合法。加入这个优化以后 BFS 就可以通过本题的数据了，但还有改进的空间。

其次是换一个算法，例如双向广度优先搜索^①。这种算法在前面并没有介绍，但是对于“暴力搜索”这样的非常规算法来说，并不一定要严格遵守所谓的“标准方法”。例如，提到“双向广度优先算法”，可以“想当然”地设计出这样的算法：正着搜索一层，反着搜索一层，然后继续这样交替下去，直到两层中出现相同的状态，读者不妨一试。

本题非常经典，强烈推荐读者编写程序。

7.6 迭代加深搜索

迭代加深搜索是一个应用范围很广的算法，不仅可以像回溯法那样找一个解，也可以像状态空间搜索那样找一条路径。下面先举一个经典的例子。

埃及分数问题。在古埃及，人们使用单位分数的和（即 $1/a$ ， a 是自然数）表示一切有理数。例如， $2/3=1/2+1/6$ ，但不允许 $2/3=1/3+1/3$ ，因为在加数中不允许有相同的。

对于一个分数 a/b ，表示方法有很多种，其中加数少的比加数多的好，如果加数个数相同，则最小的分数越大越好。例如， $19/45=1/5+1/6+1/18$ 是最优方案。

输入整数 a, b ($0 < a < b < 500$)，试编程计算最佳表达式。

样例输入：

495 499

^① 还有一个不错的候选算法是 A*，可惜超出了本书的范围，有兴趣的读者可以自行搜索相关资料。

样例输出:

Case 1: 495/499=1/2+1/5+1/6+1/8+1/3992+1/14970

【分析】

这道题目理论上可以用回溯法求解，但是解答树非常“恐怖”——不仅深度没有明显的上界，而且加数的选择在理论上也是无限的。换句话说，如果用宽度优先遍历，连一层都扩展不完（因为每一层都是无限大的）。

解决方案是采用迭代加深搜索（iterative deepening）：从小到大枚举深度上限 $\max d$ ，每次执行只考虑深度不超过 $\max d$ 的结点。这样，只要解的深度有限，则一定可以在有限时间内枚举到。

提示 7-17：对于可以用回溯法求解但解答树的深度没有明显上限的题目，可以考虑使用迭代加深搜索（iterative deepening）。

深度上限 $\max d$ 还可以用来“剪枝”。按照分母递增的顺序来进行扩展，如果扩展到 i 层时，前 i 个分数之和为 c/d ，而第 i 个分数为 $1/e$ ，则接下来至少还需要 $(a/b - c/d)/(1/e)$ 个分数，总和才能达到 a/b 。例如，当前搜索到 $19/45 = 1/5 + 1/100 + \dots$ ，则后面的分数每个最大为 $1/101$ ，至少需要 $(19/45 - 1/5)/(1/101) = 23$ 项总和才能达到 $19/45$ ，因此前 22 次迭代是根本不会考虑这棵子树的。这里的关键在于：可以估计至少还要多少步才能出解。

注意，这里的估计都是乐观的，因为用了“至少”这个词。说得学术一点，设深度上限为 $\max d$ ，当前结点 n 的深度为 $g(n)$ ，乐观估价函数为 $h(n)$ ，则当 $g(n) + h(n) > \max d$ 时应该剪枝。这样的算法就是 IDA*。当然，在实战中不需要严格地在代码里写出 $g(n)$ 和 $h(n)$ ，只需要像刚才那样设计出乐观估价函数，想清楚在什么情况下不可能在当前的深度限制下出解即可。

提示 7-18：如果可以设计出一个乐观估价函数，预测从当前结点至少还需要扩展几层结点才有可能得到解，则迭代加深搜索变成了 IDA* 算法。

本题的主框架就是一个简单循环：

```
int ok = 0;
for(maxd = 1; ; maxd++) {
    memset(ans, -1, sizeof(ans));
    if(dfs(0, get_first(a, b), a, b)) { ok = 1; break; }
}
```

其中 $\text{get_first}(a, b)$ 是满足 $1/c \leq a/b$ 的最小 c 。迭代加深搜索过程如下（约分的原理详见第 10 章）：

```
//如果当前解 v 比目前最优解 ans 更优，更新 ans
bool better(int d) {
    for(int i = d; i >= 0; i--) if(v[i] != ans[i]) {
        return ans[i] == -1 || v[i] < ans[i];
    }
}
```

```

    }
    return false;
}

//当前深度为 d, 分母不能小于 from, 分数之和恰好为 aa/bb
bool dfs(int d, int from, LL aa, LL bb) {
    if(d == maxd) {
        if(bb % aa) return false; //aa/bb 必须是埃及分数
        v[d] = bb/aa;
        if(better(d)) memcpy(ans, v, sizeof(LL) * (d+1));
        return true;
    }
    bool ok = false;
    from = max(from, get_first(aa, bb)); //枚举的起点
    for(int i = from; ; i++) {
        //剪枝: 如果剩下的 maxd+1-d 个分数全部都是 1/i, 加起来仍然不超过 aa/bb, 则无解
        if(bb * (maxd+1-d) <= i * aa) break;
        v[d] = i;
        //计算 aa/bb - 1/i, 设结果为 a2/b2
        LL b2 = bb*i;
        LL a2 = aa*i - bb;
        LL g = gcd(a2, b2); //以便约分
        if(dfs(d+1, i+1, a2/g, b2/g)) ok = true;
    }
    return ok;
}

```

例题 7-10 编辑书稿 (Editing a Book, UVa 11212)

你有一篇由 n ($2 \leq n \leq 9$) 个自然段组成的文章, 希望将它们排列成 $1, 2, \dots, n$ 。可以用 Ctrl+X (剪切) 和 Ctrl+V (粘贴) 快捷键来完成任务。每次可以剪切一段连续的自然段, 粘贴时按照顺序粘贴。注意, 剪贴板只有一个, 所以不能连续剪切两次, 只能剪切和粘贴交替。

例如, 为了将 {2,4,1,5,3,6} 变为升序, 可以剪切 1 将其放到 2 前, 然后剪切 3 将其放到 4 前。再如, 对于排列 {3,4,5,1,2}, 只需一次剪切和一次粘贴即可——将 {3,4,5} 放在 {1,2} 后, 或者将 {1,2} 放在 {3,4,5} 前。

【分析】

本题是典型的状态空间搜索问题, “状态”就是 $1 \sim n$ 的排列, 初始状态是输入, 终止状态是 $1, 2, 3, \dots, n$ 。因为 $n \leq 9$, 排列最多有 $9! = 362880$ 个。虽然这个数字不算大, 但是每个状态的后继状态也比较多 (有很多剪切和粘贴的方式), 所以仍有超时的危险。比赛时很多选手使用了一些“加速策略”。

策略 1: 每次只剪切一段连续的数字。例如, 不要剪切 2 4 这样数字不连续的片段。

策略 2: 假设剪切片段的第一个数字为 a , 最后一个数字为 b , 要么把这个片段粘贴到 $a-1$ 的下一个位置, 要么粘贴到 $b+1$ 的前一个位置。

策略 3: 永远不要“破坏”一个已经连续排列的数字片段。例如, 不能把 1 2 3 4 中的 2 3 剪切出来。

3 种策略都能缩小状态空间, 但它们并不都是正确的。很多程序都无法得到“5 4 3 2 1”的正确结果 (答案是 3 步而不是 4 步: $5\ 4\ 3\ 2\ 1 \rightarrow 3\ 2\ 5\ 4\ 1 \rightarrow 3\ 4\ 1\ 2\ 5 \rightarrow 1\ 2\ 3\ 4\ 5$), 读者不妨自行验证上面的 3 种策略是否可以得到这组数据的正确答案。

本题可以用 IDA* 算法求解。不难发现 $n \leq 9$ 时最多只需要 8 步, 因此深度上限为 8。IDA* 的关键在于启发函数。考虑后继不正确的数字个数 h , 可以证明每次剪切时 h 最多减少 3, 因此当 $3d+h > 3\max d$ 时可以剪枝, 其中 d 为当前深度, $\max d$ 为深度限制^①。

如何证明每次剪切时 h 最多减少 3 呢? 如图 7-19 所示, 因为最多只有 3 个数字的后继数字发生了改变 (即图中的 a, b, c), h 自然最多减少 3。



图 7-19 h 最多减少 3

7.7 竞赛题目选讲

本章的篇幅不少, 但实际上介绍的算法很有系统性, 并不杂乱。这里先把这些算法和常见解决问题的思路总结一下, 然后选讲一些例题,

直接枚举。例如, 类似“1~ n 的整数中有多少个满足……”, “输入一个长度为 n 的序列, 有多少个连续子序列满足……”的问题都可以用直接枚举法。枚举法可以解决问题, 但是效率不一定足够高。第 8 章中将详细讨论算法效率的分析方法。

枚举子集和排列。 n 个元素的子集有 2^n 个, 可以用递归的方法枚举 (前面介绍的增量法和位向量法都属于递归枚举), 也可以用二进制的方法枚举。递归法的优点在于效率高, 方便剪枝, 缺点在于代码比较长。一般来说, 当 n 很小 (如 $n \leq 15$) 时, 会使用二进制的方式枚举。

n 个不同元素的全排列有 $n!$ 个。除了用递归的方法枚举之外, 还可以用 STL 的 `next_permutation` 来枚举, 它也适用于有重复元素的情形。

回溯法。简单地说, 回溯法几乎就是递归枚举, 只是多了一条: 违反题目要求时及时终止当前递归过程, 即回溯 (backtracking)。回溯法最经典的题目就是八皇后问题, 这个问题也常常被作为“判断有没有学过回溯法”的依据。7.4 节的几个例题非常经典, 覆盖了回溯法的几个常见话题: 搜索对象的选取 (天平难题)、最优性剪枝 (带宽), 以及减少

^① 此处故意没有用前面介绍的 $h(s)$ 、 $g(s)$ 等记号。事实上, 经常采用这种直观的方式来思考, 而不去理会那些记号。

无用功(困难的串)。

状态空间搜索。从本质上讲,状态空间搜索算法和图算法的相似度比较大,但是图往往是“隐式”给出,所以这些算法又称“隐式图搜索”或者“产生式系统”^①。如果仔细品味前面《八数码问题》的解法,可以发现这个解法其实就是一个普通的 BFS 加上了“结点查找表”。前面介绍了 3 种方法实现结点查找表,各有用武之地。建议读者先熟练掌握后面两种(哈希表和 STL 集合),待学习完第 10 章后再尝试使用第一种方法(一一映射,或称“完美哈希”)。这些方法不仅能加快状态空间搜索的速度,还能给其他算法加速。第 8 章和第 9 章中将继续讨论这个问题。另外,双向广度优先搜索和 A* 等算法也有各自的用武之地,虽然限于篇幅未加介绍,但是笔者鼓励大家花一些时间搜索相关资料,并加以学习。例题中的“万圣节后的早晨”就是一处很好的“试验田”。

迭代加深搜索。本章最后介绍了迭代加深搜索。这是一个长期以来被“低估”了的算法,可以用来解决很多看起来更适合用 BFS 或者回溯法解决的问题,埃及分数问题就是一个绝好的例子,而例题“编辑书稿”也非常经典。

例题 7-11 宝箱 (Zombie's Treasure Chest, Shanghai 2011, UVa12325)

你有一个体积为 N 的箱子和两种数量无限的宝物。宝物 1 的体积为 S_1 , 价值为 V_1 ; 宝物 2 的体积为 S_2 , 价值为 V_2 。输入均为 32 位带符号整数。你的任务是计算最多能装多大价值的宝物。例如, $n=100$, $S_1=V_1=34$, $S_2=5$, $V_2=3$, 答案为 86, 方案是装两个宝物 1, 再装 6 个宝物 2。每种宝物都必须拿非负整数个。

【分析】

最容易想到的方法是:枚举宝物 1 的个数,然后尽量多拿宝物 2。这样做的时间复杂度为 $O(N/S_1)$, 当 N 和 S_1 相差非常悬殊时效率很低。当然,如果 N/S_2 很小时可以改成枚举宝物 2 的个数,所以这个方法不奏效的条件是: S_1 和 S_2 都很小,而 N 很大。

幸运的是, S_1 和 S_2 都很小时,有另外一种枚举法 B: S_2 个宝物 1 和 S_1 个宝物 2 的体积相等,而价值分别为 $S_2 \cdot V_1$ 和 $S_1 \cdot V_2$ 。如果前者比较大,则宝物 2 最多只会拿 S_1-1 个(否则可以把 S_1 个宝物 2 换成 S_2 个宝物 1); 如果后者比较大,则宝物 1 最多只会拿 S_2-1 个。不管是哪种情况,枚举量都只有 S_1 或者 S_2 。

这样,就得到了一个比较“另类”的分类枚举算法:

当 N/S_1 比较小时枚举宝物 1 的个数,时间复杂度为 $O(N/S_1)$, 否则,当 N/S_2 比较小时枚举宝物 2 的个数,时间复杂度为 $O(N/S_2)$, 否则说明 S_1 和 S_2 都比较小,执行枚举法 B,时间复杂度为 $O(\max\{S_1, S_2\})$ 。

例题 7-12 旋转游戏 (The Rotation Game, Shanghai 2004, UVa1343)

如图 7-20 所示形状的棋盘上分别有 8 个 1、2、3, 要往 A~H 方向旋转棋盘, 使中间 8 个方格数字相同。图 7-20 (a) 进行 A 操作后变为图 7-20 (b), 再进行 C 操作后变为图 7-20 (c), 这正是一个目标状态(因为中间 8 个方格数字相同)。要求旋转次数最少。如果有多解, 操作序列的字典序应尽量小。

^① 这个术语多用在传统人工智能书籍中, 虽有一些描述上的差别, 但本质相同。

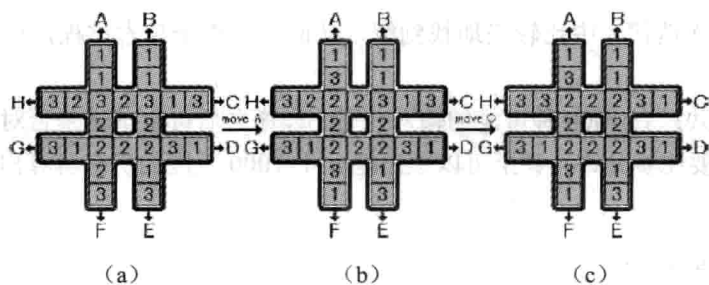


图 7-20 旋转游戏示意图

【分析】

本题是一个典型的状态空间搜索问题，可惜如果直接套用八数码问题的框架会超时。为什么？学完第 10 章的组合计数部分后会知道：8 个 1、8 个 2、8 个 3 的全排列个数为 $24!/(8!*8!*8!)=9465511770$ 。换句话说，最坏情况下最多要处理这么多结点！

解决方法很巧妙：本题要求的是中间 8 个数字相同，即 8 个 1 或者 8 个 2 或者 8 个 3。因此可以分 3 次求解。当目标是“中间 8 个数字都是 1”时，2 和 3 就没有区别了（都是“非 1”），因此状态总数变成了 8 个 1，16 个“非 1”的全排列个数，即 $24!/(8!*16!)=735471$ ，在可以接受的范围了^①。

例题 7-13 快速幂计算 (Power Calculus, ACM/ICPC Yokohama 2006, UVa1374)

输入正整数 n ($1 \leq n \leq 1000$)，问最少需要几次乘除法可以从 x 得到 x^n ？例如， x^{31} 需要 6 次： $x^2=x*x$ ， $x^4=x^2*x^2$ ， $x^8=x^4*x^4$ ， $x^{16}=x^8*x^8$ ， $x^{32}=x^{16}*x^{16}$ ， $x^{31}=x^{32}/x$ 。计算过程中 x 的指数应当总是正整数（如 $x^{-3}=x/x^4$ 是不允许的）。

【分析】

这个题有一点“埃及分数”的味道，可以考虑迭代加深搜索。当前状态是已经得到的指数集合，操作是任选两个数进行加法和减法，并且不能产生重复的数，如图 7-21 所示。

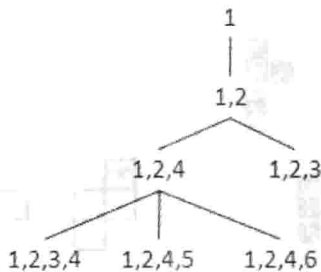


图 7-21 快速幂计算示意图

沿用之前的符号， d 表示当前深度， maxd 表示深度上限，则如果当前序列最大的数乘以 $2^{\text{maxd}-d}$ 之后仍小于 n ，则剪枝（想一想，为什么）。另外，为了尽快接近目标，不应该“任选”两个数，而应该先选较大的数，并且先试加法再试减法^②。这样做可以在最后一次迭代

^① 一般来说，状态总数不超过 10^6 时都在可接受范围内。不过这只是一般规律，还要具体问题具体分析。

^② 这种技巧称为结点排序 (node ordering)。

（即找到解的那次迭代）中比较快地找到解，从而终止整个搜索过程，而不需要等整个解答树扩展完毕。

因为题目一共只有 1000 种可能的输入，写完程序之后可以试试是否对所有输入都能足够快地出解。只要比赛允许，甚至可以预先把 $n=1\sim 1000$ 范围的所有解算出来，输出成如下源代码：

```
#include<cstdio>
int answer[] = {0, 0, 1, ...}; //answer[1]=0, answer[2]=1, ...
int main() {
    int n;
    while(scanf("%d", &n) == 1 && n) printf("%d\n", answer[n]);
    return 0;
}
```

这样的技巧俗称“打表”。本题还有一些常见的优化，例如，限制减法的次数（实际上大部分时候都是最大的数乘以 2），或者限制超过 n 的数的个数（事实上，可以证明最多有一个数需要超过 n ），读者不妨一试。

例题 7-14 网格动物 (Lattice Animals, ACM/ICPC NEERC 2004, UVa1602)

输入 n, w, h ($1 \leq n \leq 10, 1 \leq w, h \leq n$)，求能放在 $w \times h$ 网格里的不同的 n 连块的个数（注意，平移、旋转、翻转后相同的算作同一种）。例如， 2×4 里的 5 连块有 5 种（第一行），而 3×3 里的 8 连块有以下 3 种（第二行），如图 7-22 所示。

【分析】

本题看上去没有什么好办法，只能用回溯法求解。如何求解呢？首先需要确定搜索对象。因为要求各个格子连通，所以可以把“连通块”作为搜索对象，每次枚举一个位置，然后放一个新的块，最后判重，如图 7-23 所示。

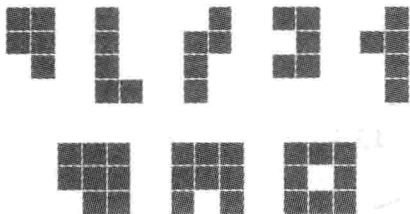


图 7-22 网格动物例题示意图

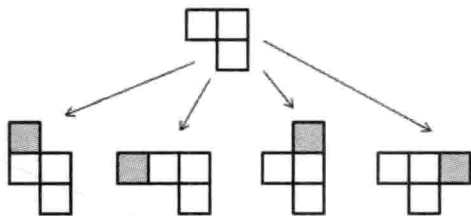


图 7-23 回溯法求解

需要注意的是，如果采用最简单的写法，每个 n 连块都会被重复枚举很多次（想一想，为什么）。也可以用前面介绍过的方法判重，但实际上有办法确保每个 n 连块恰好被枚举一次，由 Redelmeier 发现，有兴趣的读者可以自行研究^①。

本题非常经典，强烈建议读者编写程序。

^① 可以参考 en.wikipedia.org/wiki/Polyomino。

例题 7-15 破坏正方形 (Square Destroyer, ACM/ICPC Taejon 2001, UVa1603)

有一个火柴棍组成的正方形网格，每条边有 n 根火柴，共 $2n(n+1)$ 根。从上到下、从左到右给各个火柴编号，如图 7-24 (a) 所示。现在拿走一些火柴，问在剩下的火柴中，至少还要拿走多少根火柴才能破坏所有正方形？例如，在图 7-24 (b) 中，拿掉 3 根火柴就可以破坏掉仅有的 5 个正方形。

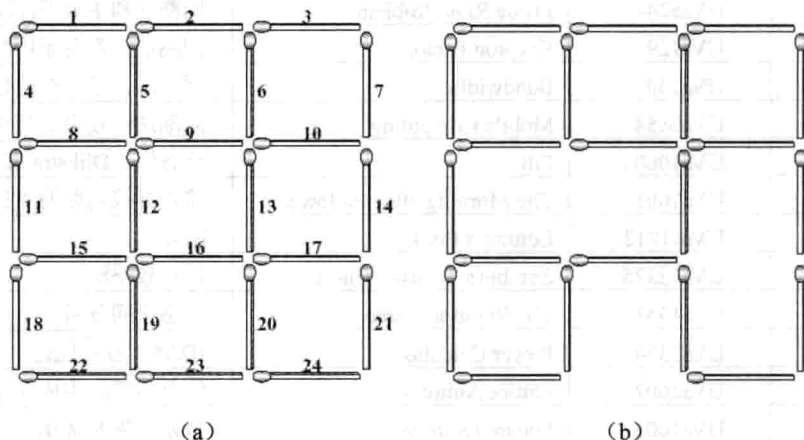


图 7-24 破坏正方形示意图

【分析】

不难想到用迭代加深搜索作为主算法框架。搜索对象有两种：(1) 每次考虑一个没有被破坏的正方形，在边界上找一根火柴拿掉；(2) 每次找一个至少能破坏一个正方形的火柴，然后拿掉。两种方法各有不同的优化方法：

搜索对象是正方形。应先考虑小正方形，再考虑大正方形，因为破坏完小正方形之后，很多大正方形已经被破坏了，但是反过来却不一定。还可以加入最优性剪枝，即把每个正方形看成一个顶点，有公共火柴的正方形连一条边，则每个连通分量至少要拿走一根火柴。

搜索对象是火柴。应先搜索能破坏尽量多正方形的火柴。这需要计算出待考虑的每根火柴可以破坏掉多少个正方形，从大到小排序为 $d[1], d[2], d[3], \dots$ 。当 $d[1]=1$ 时即可停止搜索，因为此时可以直接计算出还需要的火柴个数（想一想，为什么）。这个 d 数组也可以用于最优性剪枝，找到最小的 i ，使得 $d[1]+d[2]+\dots+d[i] \geq k$ （其中 k 为还剩的正方形个数），则至少还要 i 根火柴。

值得一提的是：本题还可以用经典的 DLX 算法解决。该算法超出了本章的范围，但在《算法竞赛入门经典——训练指南》中有详细叙述。

7.8 训练参考

前面已经提到过，本章介绍的算法比较有系统性，因此也没有选择太多的例题。建议读者独立完成所有例题。本章例题列表及说明如表 7-3 所示。

表 7-3 例题列表

类 别	题 号	题目名称（英文）	备 注
例题 7-1	UVa725	Division	选择合适的枚举对象
例题 7-2	UVa11059	Maximum Product	枚举连续子序列
例题 7-3	UVa10976	Fractions Again?!	缩小枚举范围
例题 7-4	UVa524	Prime Ring Problem	回溯法和生成-测试法的比较
例题 7-5	UVa129	Krypton Factor	回溯法；避免无用判断
例题 7-6	UVa140	Bandwidth	回溯法；最优性剪枝
例题 7-7	UVa1354	Mobile Computing	回溯法；枚举二叉树
例题 7-8	UVa10603	Fill	状态图，Dijkstra 算法
例题 7-9	UVa1601	The Morning after Halloween	路径寻找问题的“试验田”
例题 7-10	UVa11212	Editing a Book	IDA*
例题 7-11	UVa12325	Zombie's Treasure Chest	两种枚举法
例题 7-12	UVa1343	The Rotation Game	状态空间分析
例题 7-13	UVa1374	Power Calculus	IDA*，各种优化
例题 7-14	UVa1602	Lattice Animals	经典问题：生成 n 连块
例题 7-15	UVa1603	Square Destroyer	搜索对象及优化

下面是本章的习题。这些题目大都具有一定的复杂性，读者可以选择自己有兴趣的 5 道题目完成。如果想达到更好的效果，建议完成至少 10 道题目。

习题 7-1 消防车（Firetruck, ACM/ICPC World Finals 1991, UVa208）

输入一个 n ($n \leq 20$) 个结点的无向图以及某个结点 k ，按照字典序从小到大顺序输出从结点 1 到结点 k 的所有路径，要求结点不能重复经过。

提示：要事先判断结点 1 是否可以到达结点 k ，否则会超时。

习题 7-2 黄金图形（Golygons, ACM/ICPC World Finals 1993, UVa225）

平面上有 k 个障碍点。从 $(0,0)$ 点出发，第一次走 1 个单位，第二次走 2 个单位，……，第 n 次走 n 个单位，恰好回到 $(0,0)$ 。要求只能沿着东南西北方向走，且每次必须转弯 90° （不能沿着同一个方向继续走，也不能后退）。走出的图形可以自交，但不能经过障碍点，如图 7-25 所示。

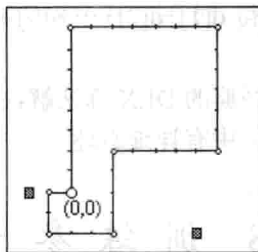


图 7-25 黄金图形示意图

输入 n, k ($1 \leq n \leq 20, 0 \leq k \leq 50$) 和所有障碍点的坐标，输出所有满足要求的移动序列（用 news 表示北、东、西、南），按照字典序从小到大排列，最后输出移动序列的总数。

习题 7-3 多米诺效应 (The Domino Effect, ACM/ICPC World Finals 1991, UVa211)

一副“双六”多米诺骨牌包含 28 张, 编号如图 7-26 所示。

Bone #	Pips	Bone #	Pips	Bone #	Pips	Bone #	Pips
1	0 0	8	1 1	15	2 3	22	3 6
2	0 1	9	1 2	16	2 4	23	4 4
3	0 2	10	1 3	17	2 5	24	4 5
4	0 3	11	1 4	18	2 6	25	4 6
5	0 4	12	1 5	19	3 3	26	5 5
6	0 5	13	1 6	20	3 4	27	5 6
7	0 6	14	2 2	21	3 5	28	6 6

图 7-26 多米诺骨牌编号

在 7*8 网格中每张牌各摆一张, 如图 7-27 所示, 左边是各个格子的点数, 右边是各个格子所属的骨牌编号。

7 x 8 grid of pips								map of bone numbers							
6	6	2	6	5	2	4	1	28	28	14	7	17	17	11	11
1	3	2	0	1	0	3	4	10	10	14	7	2	2	21	23
1	3	2	4	6	6	5	4	8	4	16	25	25	13	21	23
1	0	4	3	2	1	1	2	8	4	16	15	15	13	9	9
5	1	3	6	0	4	5	5	12	12	22	22	5	5	26	26
5	5	4	0	2	6	0	3	27	24	24	3	3	18	1	19
6	0	5	3	4	2	0	3	27	6	6	20	20	18	1	19

图 7-27 7*8 网格中骨牌摆放

输入左图, 你的任务是输出所有可能的右图。

习题 7-4 切断圆环链 (Cutting Chains, ACM/ICPC World Finals 2000, UVa818)

有 n ($n \leq 15$) 个圆环, 其中有一些已经扣在了一起。现在需要打开尽量少的圆环, 使得所有圆环可以组成一条链 (当然, 所有打开的圆环最后都要再次闭合)。例如, 有 5 个圆环, 1-2, 2-3, 4-5, 则需要打开一个圆环, 如圆环 4, 然后用它穿过圆环 3 和圆环 5 后再次闭合圆环 4, 就可以形成一条链: 1-2-3-4-5。

习题 7-5 流水线调度 (Pipeline Scheduling, UVa690)

你有一台包含 5 个工作单元的计算机, 还有 10 个完全相同的程序需要执行。每个程序需要 n ($n < 20$) 个时间片来执行, 可以用一个 5 行 n 列的保留表 (reservation table) 来表示, 其中每行代表一个工作单元 (unit0~unit4), 每列代表一个时间片, 行 i 列 j 的字符为 X 表示“在程序执行的第 j 个时间片中需要工作单元 i ”。例如, 如图 7-28 (a) 所示就是一张保留表, 其中程序在执行的第 0, 1, 2, ……个时间片中分别需要 unit0, unit1, unit2……

同一个工作单元不能同时执行多个程序, 因此若两个程序分别从时间片 0 和 1 开始执行, 则在时间片 5 时会发生冲突 (两个程序都想使用 unit0), 如图 7-28 (b) 所示。

输入一个 5 行 n ($n < 20$) 列的保留表, 输出所有 10 的任务执行完毕所需的最少时间。例如, 对于图 7-28 (a) 的保留表, 执行完 10 个程序最少需要 34 个时间片。

clock	0	1	2	3	4	5	6
unit0	X	.	.	.	X	X	.
unit1	.	X	.	.	.	.	.
unit2	.	.	X	.	.	.	.
unit3	.	.	.	X	.	.	.
unit4	.	.	.	.	.	.	X

(a)

clock	0	1	2	3	4	5	6	7
unit0	0	1	.	.	0	C	1	.
unit1	.	0	1	.	.	.	.	.
unit2	.	.	0	1	.	.	.	.
unit3	.	.	.	0	1	.	.	.
unit4	.	.	.	.	.	0	1	.

(b)

图 7-28 流水线调度示意图

习题 7-6 重叠的正方形 (Overlapping Squares, Xia'an 2006, UVa12113)

给定一个 4*4 的棋盘和棋盘上所呈现出来的纸张边缘, 如图 7-29 所示, 问用不超过 6 张 2*2 的纸能否摆出这样的形状。

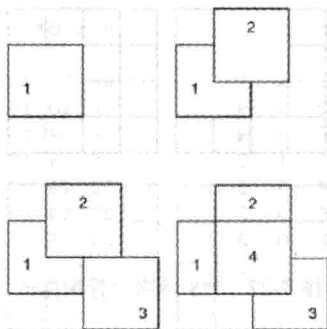


图 7-29 重叠正方形示意图

习题 7-7 埃及分数 (Egyptian Fractions (HARD version), Rujia Liu's Present 6, UVa 12558)

把 a/b 写成不同的埃及分数之和, 要求项数尽量小, 在此前提下最小的分数尽量大, 然后第二小的分数尽量大……另外有 k ($0 \leq k \leq 5$) 个数不能用作分母。例如, $k=0$ 时 $5/121=1/33+1/121+1/363$, 不能使用 33 时最优解为 $5/121=1/45+1/55+1/1089$ 。

输入保证 $2 \leq a < b \leq 876$, $\gcd(a,b)=1$, 且会挑选比较容易求解的数据。

习题 7-8 数字谜 (Digit Puzzle, ACM/ICPC Xi'an 2006, UVa12107)

给出一个数字谜, 要求修改尽量少的数, 使修改后的数字谜只有唯一解。例如, 如图 7-30 所示的两个数字谜就有唯一解。

$$7 \times \square \square = 8 \square$$

$$\square \square \times \square \square = 1 \square 1$$

图 7-30 数字谜示意图

修改指的是空格和数字可以随意替换, 但不能增删。即空格换数字、数字换空格或数字替换。数字谜中所有涉及的数必须是没有前导零的正数。输入数字谜一定形如 $a*b=c$, 其中 a 、 b 、 c 分别最多有 2、2、4 位。

输入保证有解。如果有多种修改方案，则输出字典序最小的。字典序中空格小于数字。

习题 7-9 立体八数码问题 (Cubic Eight-Puzzle, ACM/ICPC Japan 2006, UVa1604)

有 8 个立方体，按照相同方式着色 (如图 7-31 (a) 所示，相对的面总是着相同颜色)，然后以相同的朝向摆成一个 3×3 的方阵，空出一个位置 (如图 7-31 (b) 所示，空位由输入决定)。

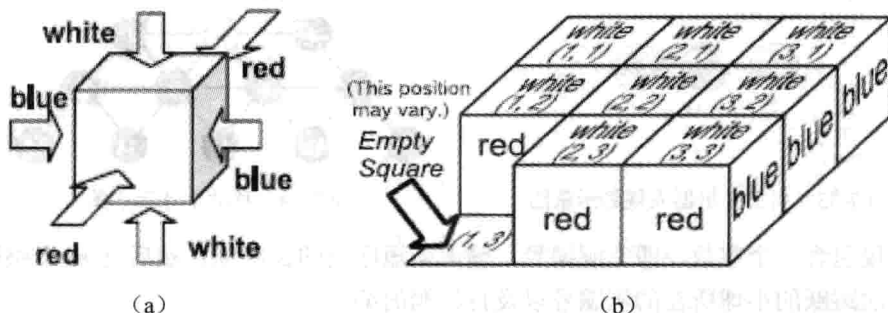


图 7-31 立体八数码问题示意图

每次可以把一个立方体“滚动”一格进入空位，使它原来的位置成为空位，如图 7-32 所示。

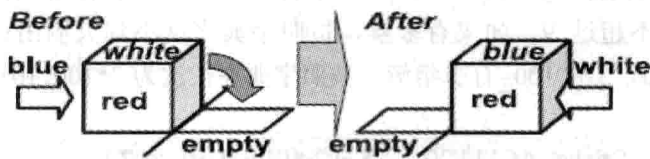


图 7-32 “滚动”后效果

你的任务是用最少的移动使得上表面呈现出指定的图案。输入空位的坐标和目标状态中上表面各个位置的颜色，输出最小移动步数。

习题 7-10 守卫棋盘 (Guarding the Chessboard, UVa11214)

输入一个 $n \times m$ 棋盘 ($n, m < 10$)，某些格子有标记。用最少的皇后守卫 (即占据或者攻击) 所有带标记的格子。

习题 7-11 树上的机器人规划 (简单版) (Planning mobile robot on Tree (EASY Version), UVa12569)

有一棵 n ($4 \leq n \leq 15$) 个结点的树，其中一个结点有一个机器人，还有一些结点有石头。每步可以把一个机器人或者石头移到一个相邻结点。任何情况下一个结点里不能有两个东西 (石头或者机器人)。输入每个石头的位置和机器人的起点和终点，求最小步数的方案。如果有多解，可以输出任意解。如图 7-33 所示， $s=1, t=5$ 时，最少需要 16 步：机器人 1-6，石头 2-1-7，机器人 6-1-2-8，石头 3-2-1-6，石头 4-3-2-1，最后机器人 8-2-3-4-5。

习题 7-12 移动小球 (Moving Pegs, ACM/ICPC Taejon 2000, UVa1533)

如图 7-34 所示，一共有 15 个洞，其中一个空着，剩下的洞里各有一个小球。每次可以让一个小球越过同一条直线上的一个或多个连续的小球，落到最近的空洞 (不能越过空洞)，

然后拿走被跳过的小球。例如，让 14 跳到空洞 5 中，则洞 9 里的小球会被拿走，因此操作之后洞 9 和 14 会变空，而 5 里面会有一个小球。你的任务是用最少的步数让整个棋盘只剩下一个小球，并且位于初始时的那个空洞中。

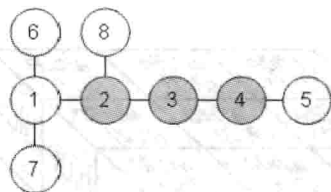


图 7-33 树上的机器人规划示意图

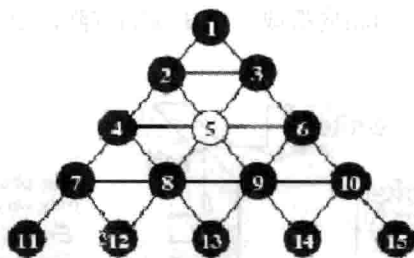


图 7-34 移动小球示意图

输入仅包含一个整数，即空洞编号，输出最短序列的长度 m ，然后是 m 个整数对，分别表示每次跳跃的小球所在的洞编号以及目标洞的编号。

习题 7-13 数字表达式 (According to Bartjens, ACM/ICPC World Finals 2000, UVa 817)

输入一个以等号结尾、前面只包含数字的表达式，插入一些加号、减号和乘号，使得运算结果等于 2000。表达式里的整数不能有前导零（例如，0100 或者 000 都是非法的），运算符都是二元的（例如， $2*-100*-10+0=$ 是非法的），并且符合通常的运算优先级法则。

输入数字个数不超过 9。如果有多解，按照字典序从小到大输出；如果无解，输出 IMPOSSIBLE。例如，2100100= 有 3 组解，按照字典序依次为 $2*100*10+0=$ 、 $2*100*10-0=$ 和 $2100-100=$ 。

习题 7-14 小木棍 (Sticks, ACM/ICPC CERC 1995, UVa 307)

乔治有一些同样长的小木棍，他把这些木棍随意地砍成几段，直到每段的长度都不超过 50。现在，他想把小木棍拼接成原来的样子，但是却忘记了自己最开始时有多少根木棍和它们的分别长度。给出每段小木棍的长度，编程帮他找出原始木棍的最小可能长度。例如，若砍完后有 4 根，长度分别为 1, 2, 3, 4，则原来可能是 2 根长度为 5 的木棍，也可能是 1 根长度为 10 的木棍，其中 5 是最小可能长度。另一个例子是：砍之后的木棍有 9 根，长度分别为 5, 2, 1, 5, 2, 1, 5, 2, 1，则最小可能长度为 6 ($5+1=5+1=5+1=2+2+2=6$)，而不是 8 ($5+2+1=8$)。

习题 7-15 最大的数 (Biggest Number, UVa11882)

在一个 R 行 C 列 ($2 \leq R, C \leq 15, R * C \leq 30$) 的矩阵里有障碍物和数字格（包含 1~9 的数字）。你可以从任意一个数字格出发，每次沿着上下左右之一的方向走一格，但不能走到障碍格中，也不能重复经过一个数字格，然后把沿途经过的所有数字连起来，如图 7-35 所示。

如图 7-35 可以得到 9784、4832145 等整数。问：能得到的最大整数是多少？

习题 7-16 找座位 (Finding Seats Again, UVa11846)

有一个 $n * n$ ($n < 20$) 的座位矩阵里坐着 k ($k \leq 26$) 个研究小组。每个小组的座位都是矩形形状。输入每个小组组长的位置和该组的成员个数，找到一种可能的座位方案。如图 7-36 所示是一组输入和对应的输出。

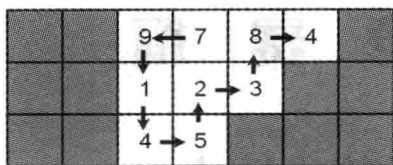


图 7-35 最大的数示意图

...4.2.	AAAAABCC
...45..	DDDDBEF
222..3.	GHIIBEF
...2..3	GHJKB EF
.24...2	LLJKBMM
...2.3.	NOJPQQQ
22..3..	NOJPRRR

图 7-36 找座位问题示意图

习题 7-17 Gokigen Naname 谜题 (Gokigen Naname, UVa11694)

在一个 $n \times n$ ($n \leq 7$) 网格中, 有些交叉点上有数字。你的任务是给每个格子画一条斜线 (一共只有 “\” 和 “/” 两种), 使得每个交叉点的数字等于和它相连的斜线条数, 且这些斜线不会构成环, 如图 7-37 所示。

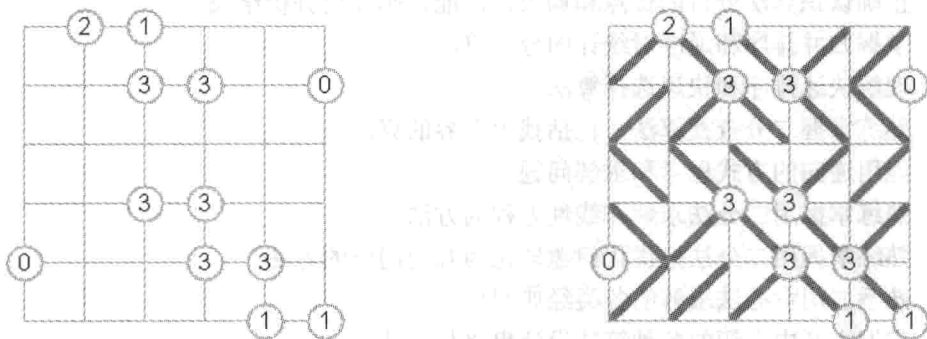


图 7-37 Gokigen Naname 谜题示意图

习题 7-18 推门游戏 (The Wall Pusher, UVa10384)

如图 7-38 所示, 从 S 处出发, 每次可以往东、南、西、北 4 个方向之一前进。如果前方有墙壁, 游戏者可以把墙壁往前推一格。如果有两堵或者多堵连续的墙, 则不能推动。另外, 游戏者也不能推动游戏区域边界上的墙。

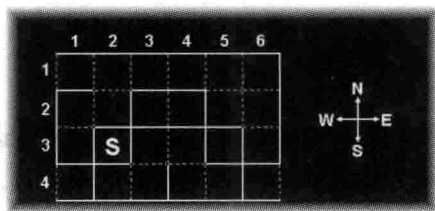


图 7-38 推门游戏示意图

用最少的步数走出迷宫 (边界处没有墙的地方就是出口)。迷宫总是有 4 行 6 列, 多解时任意输出一个移动序列即可 (用 NEWS 这 4 字符表示移动方向)。